Numerical Methods

Lecture 1: Interpolation and curve fitting

1. 基础概念

1.1 Discerete data(离散数据)

离散数据就是一组数据点，每个数据点由两个值组成，一个是x值，一个是y值。离散数据可以表示为(x1,y1),(x2,y2),...,(xn,yn)。不论是interpolation(插值算法)还是curve fitting（拟合算法），都是针对discrete data（离散数据）进行的，目的都是要使得其连续化，也就是说从散点图（plot）生成曲线图（line）

1.2 Different between interpolation and curve fitting

理解不同算法的原来，用途，优劣是一个考试中的常见问题

1.2.1 Interpolation (内推法)

Interpolation is like filling in the gaps between known points using the data from **nearby** known points. It is commonly used in mapping and GIS (地理信息系统).

- Since interpolation just fills in the gaps without considering the overall trend or internal relationships, it works well **between** known points but not so well **outside** the known points.
- The result of interpolation **always** passes through the known points. 插值/内插 就是用附近的已知点简单填空，所以插值的结果一定经过已知点，不考虑内在关联性

1.2.2 Extraploation (外推法)

Extrapolation is a method used to estimate values outside the range of known data points by extending the trend observed in the known data. Extrapolation does not consider the overall trend or internal relationships; it only uses the trend from a few nearby points to estimate values.

- Extrapolation is used when the known data points are close to each other and the trend is consistent. However, if the known data points are far apart or the trend is inconsistent, extrapolation may not be accurate.
- The result of extrapolation **still** passes through the known points. 外推法 用附近的已知点预测外部的数据，所以外推的结果一定经过已知点，但是因为没有考虑到内在关联性，只有离得不太远的时候才灵

1.2.3 Curve Fitting (拟合)

Curve fitting focuses on the trend and the **relationship** between (X) and (Y). It is commonly used when studying relationships or making predictions. Ususally least square method is used to fit the curve.一般fit的方法就是让误差（曲线和已知点在同一个X的插值）的平方加起来最小。

- Because curve fitting focuses on the trend, it performs better than interpolation **outside** the known points. This is based on the overall trend rather than just filling in gaps.
- The result of curve fitting **does not** necessarily pass through the known points. 拟合 用整体趋势来预测，所以拟合的结果不一定经过已知点，但是因为考虑了内在关联性，所以确定性更强，如果是离已知点比较远效果比插值算法要好

1.3 Degree of polynomial of Interpolation 确定插值多项式的阶数

用几次函数去做插值和拟合是一个考试中的常见问题 The degree of a polynomial is defined as the highest power of the variable in the polynomial expression. For example, in the polynomial $2x^3 + 3x^2 - 4x + 5$, the degree is 3. To determine the degree of the polynomial that interpolates your data, you need to consider the number of data points you have. If you have (n + 1) data points, the polynomial that interpolates these points will be of degree. For example, if you have 4 data points, the interpolating polynomial will be of degree 3. 简单来讲就是如果有n个点，那么插值多项式的阶数就是n-1。

1.4 Situations Where It Might Not Agree:

There are situations where the degree of the polynomial might not be the best choice:

- **Overfitting**: If the degree of the polynomial is too high relative to the number of data points, the polynomial might fit the data very well but will likely perform poorly when used for extrapolation or even interpolation outside the range of the data points. This is because high-degree polynomials can oscillate wildly between data points.
- **Noise in Data**: If the data contains noise or errors, a high-degree polynomial might capture this noise rather than the underlying trend, leading to a poor model.
- **Computational Complexity**: High-degree polynomials can be computationally expensive to work with and may lead to numerical instability, especially when dealing with large datasets or floating-point arithmetic.

  In these cases, a lower-degree polynomial or a different type of model (such as a non-polynomial function) might be more appropriate to avoid overfitting and ensure better generalization to new data.

2 代码实现

2.1 Lagrange Polynomial Interpolation（拉格朗日多项式插值）

很多时候会问你degree，这里就是n个数字的话，degree就是n-1。

In [ ]:
```
'''第一步把你需要的包import进来，报错的话缺什么就import什么'''
import numpy as np
import scipy.interpolate
import matplotlib.pyplot as plt

'''第二步把数据按照格式写好，然后一定是针对散点图，顺序是一一对应的'''

time = [0,4,8,12,16,20,24,28,32,36]
shot = [-0.021373, -0.024578, -0.023914, -0.018227, -0.00781, 0.005602, 0

'''第三步这里就是计算拉格朗日多项式的函数表达式，lp就是每一项的系数(y=a+bx+c*x^2中的
从下面的结果可以看到lp的项数正好和我们有几组数据是一样的都是10，对应的degree就是9。''

# Create the Lagrange polynomial for the given points.
lp = scipy.interpolate.lagrange(time, shot)

'''第四步就是把函数变成一条线，虽然说图上是线但是在数据上其实也是散点。
所以这里我们需要先生成一个这条线的x的列表time2，numpy.linspace(0, 36, 200)就是生成

# Evaluate this fuction at a high resolution so that we can get a smooth
time2 = np.linspace(0, 36, 200)

'''然后就是把这条线的散点图算出来，lp()这个函数只要输入x就可以得到对应的y值，所以lp(t
然后就是plot，pylab.plot()就是画图，第一个参数是x的值，第二个参数是y的值，第三个参数

plt.plot(time2, lp(time2), 'b', label='Lagrange')

'''第五步把原始的数据点也画上去'''

# Overlay raw data
plt.plot(time, shot, 'k*', label='raw data')

'''最后标上横纵坐标标题和图例'''

# Add a legend
plt.xlabel('time')#
plt.ylabel('shot')
plt.legend(loc='best')

plt.show()
```

2.2 Least Square Curve Fit

一般是用least squre的方法去fit，可能还会让你求一下error。 实际上numpy和scipy都可以做，考试答案里用的numpy。

In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt

'''第一步也是把数据写成特定格式，这里是一个array，类似成一个一维的矩阵，其实就是当作li
# consider the above example data again
xi=np.array([0.5,2.0,4.0,5.0,7.0,9.0])
```

```
yi=np.array([0.5,0.4,0.3,0.1,0.9,0.8])

'''第二步：用 np.polyfit(xi, yi, degree) 做 polynomial fit（多项式拟合）。
- 第一个参数是 x 数据（xi），第二个是 y 数据（yi）。
- 第三个参数 degree 是多项式的次数（必须是非负整数：0, 1, 2, 3...）

不同 degree 对应的模型形式：
- degree = 0：constant，y = a
- degree = 1：linear，y = a*x + b
- degree = 2：quadratic，y = a*x^2 + b*x + c
- degree = 3：cubic，y = a*x^3 + b*x^2 + c*x + d

np.polyfit 的输出是 coefficients（系数数组），长度 = degree + 1，
并且顺序是“从最高次到最低次”（例如 degree=2 返回 [a, b, c] 表示 a*x^2 + b*x + c)

# Calculate coefficients of polynomial degree 0 - ie a constant value.
poly_coeffs = np.polyfit(xi, yi, 0)

'''把 coefficients 传给 np.poly1d(coeffs)，就能得到一个可调用的 polynomial fur
之后直接用 p(x) 来算对应的 y 值、画曲线。'''

# Construct a polynomial function which we can use to evaluate for arbitr
p0 = np.poly1d(poly_coeffs)

# Fit a polynomial degree 1 - ie a straight line.
poly_coeffs=np.polyfit(xi, yi, 1)
p1 = np.poly1d(poly_coeffs)

# Quadratic
poly_coeffs=np.polyfit(xi, yi, 2)
p2 = np.poly1d(poly_coeffs)

# Cubic
poly_coeffs=np.polyfit(xi, yi, 3)
p3 = np.poly1d(poly_coeffs)

''' 画图的参数其实是不怎么需要改的'''

# set up figure
fig = plt.figure(figsize=(8, 6))
ax1 = fig.add_subplot(111)
ax1.margins(0.1)

'''第三步还是需要先生成一个线的x列表，然后带入函数算出Y值，然后plot出来
np.linspace(0.4, 9.1, 100)这个函数就是生成一个从0.4到9.1的100个点，然后返回一个列
p0(x)这个函数就是带入x值得到对应的y值
ax1.plot(x, p0(x), 'k', label='Constant')这个函数就是画图，x是x轴的值，p0(x)是

x = np.linspace(0.4, 9.1, 100)

ax1.plot(x, p0(x), 'k', label='Constant')
ax1.plot(x, p1(x), 'b', label='Linear')
ax1.plot(x, p2(x), 'r', label='Quadratic')
ax1.plot(x, p3(x), 'g', label='Cubic')

'''最后就是把原始点画上去再加上标题和图例，和之前讲的一样，画原始点的函数是ax.plot(xi

def plot_raw_data(xi, yi, ax):
        """plot x vs y on axes ax,
        add axes labels and turn on grid
```

```
        """
        ax.plot(xi, yi, 'ko', label='raw data')
        ax.set_xlabel('$x$', fontsize=16)
        ax.set_ylabel('$y$', fontsize=16)
        ax.grid(True)

# Overlay raw data
plot_raw_data(xi, yi, ax1)

ax1.legend(loc='best', fontsize = 12)
ax1.set_title('Polynomial approximations of differing degree', fontsize=1

plt.show()
```

2.2.1 Square of Difference

有时候会让你计算Square of Difference，是error/difference的平方和。 error/difference就是真实值和curve在y上的差。 所以计算Square of Difference就是计算每一个已知点上真实值和curve在y上的差的平方，再全部加起来

In [ ]:
```
'''下面就是计算error了
广义上的error就是p(x)-y的平方和，或者说计算每一个已知点上真实值和curve在y上的差的平方
这个数字越小说明fit的越好，fit就是为了降低这个数字'''

def sqr_error(p, xi, yi):
                """"function to evaluate the sum of square of errors"""
                # first compute the square of the differences
                diff2 = (p(xi)-yi)**2
                # and return their sum
                return diff2.sum()

print("Number of points to fit: ", np.size(xi))
# in this example we have 6 pieces of information, to fit a polynomial
# that exactly goes through these points we need 6 unknowns, or free
# parameters, to choose in the polynomial.  [Too few and we won't be able
# to fit the data exaclty, and too many would just be a waste.  Cf. over-
# and under-determined systems.]
# A 5th order polynomial has 6 free parameters (all the powers up to 5,
# including 0).
# So calling polyfit to fit a polynomial of degree 5 (size(x)-1) should
# fit the data exactly (repeat below with size(x)-2 etc, and size(x) and
# above to convince yourself of this).
# Let's check the errors up to this point:

# Let's set up some space to store all the polynomial coefficients
# there are some redundancies here, and we have assumed we will only
# consider polynomials up to degree N

'''下面这里是要计算不同degree下的square of the difference
一开始先创建了一个全是0的N*N的矩阵poly_coeffs = np.zeros((N, N))，用来储存不同deg

N = 6
poly_coeffs = np.zeros((N, N))

'''然后写了一个循环，便利了0到N-1，计算了不同degree下的多项式系数，并储存在poly_coe
注意一下下面的语法，[i, :(i+1)]，这里表示的是从第i行开始，到第i+1行结束，不包括第i+1

for i in range(N):
```

```
                poly_coeffs[i, :(i+1)] = np.polyfit(xi, yi, i)

print('poly_coeffs = \n{}'.format(poly_coeffs))

'''然后又写了一个循环用上面计算出来的多项式系数来计算不同degree下square of the dif
np.poly1d(poly_coeffs[i, :(i+1)])这里表示的是用第i行来创建一个多项式，然后sqr_er

# and now compute the errors
for i in range(N):
                p = np.poly1d(poly_coeffs[i, :(i+1)])
                print('square of the difference between the data and the
                                        'polynomial of degree {0:1d} = {1
```

```
Number of points to fit:  6
poly_coeffs =
[[ 0.5         0.          0.          0.          0.          0.        ]
 [ 0.0508044   0.26714649  0.          0.          0.          0.        ]
 [ 0.02013603 -0.13983999  0.55279339  0.          0.          0.        ]
 [-0.00552147  0.09889271 -0.43193108  0.75909819  0.          0.        ]
 [-0.00420655  0.07403681 -0.38492428  0.59251888  0.27906056  0.        ]
 [-0.00301599  0.06536037 -0.49614427  1.59623195 -2.08266478  1.2003016
6]]
square of the difference between the data and the polynomial of degree 0 =
4.60000000e-01
square of the difference between the data and the polynomial of degree 1 =
3.32988992e-01
square of the difference between the data and the polynomial of degree 2 =
1.99478242e-01
square of the difference between the data and the polynomial of degree 3 =
1.57303437e-01
square of the difference between the data and the polynomial of degree 4 =
4.69232378e-02
square of the difference between the data and the polynomial of degree 5 =
2.02576129e-26
```

2.3 Extrapalotion

实际上extrapalotion的做法和curve fit是一模一样的，所以就用上面的代码. 如果要求必须要过每一个点的话，degree就是点的数量减一，没有要求的话和curve fit一样。

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt

'''整理数据格式'''
# consider the above example data again
xi=np.array([0.5,2.0,4.0,5.0,7.0,9.0])
yi=np.array([0.5,0.4,0.3,0.1,0.9,0.8])

'''如果要过每一个点的话degree就是点的数量减一，这里用len(xi)-1计算了一下'''
degree = len(xi) - 1

'''用np.polyfit计算多项式的系数，返回的poly_coeffs就是多项式的系数.
pn就是多项式的函数,输入x返回y'''

poly_coeffs = np.polyfit(xi, yi, degree)
pn = np.poly1d(poly_coeffs)

'''画图参数不用动'''
```

```
# set up figure
fig = plt.figure(figsize=(8, 6))
ax1 = fig.add_subplot(111)
ax1.margins(0.1)

'''生成curve的x值'''

x = np.linspace(0.4, 9.1, 100)

'''生成curve的y值并且plot'''

ax1.plot(x, pn(x), 'k', label='Extraploation')

'''画上已知点'''

def plot_raw_data(xi, yi, ax):
    """plot x vs y on axes ax,
    add axes labels and turn on grid
    """
    ax.plot(xi, yi, 'ko', label='raw data')
    ax.set_xlabel('$x$', fontsize=16)
    ax.set_ylabel('$y$', fontsize=16)
    ax.grid(True)

# Overlay raw data
plot_raw_data(xi, yi, ax1)

ax1.legend(loc='best', fontsize = 12)
ax1.set_title('Extraploation', fontsize=16);

plt.show()
```

In [ ]: